

Desenvolvimento *Web* III

Tecnologia em Análise e Desenvolvimento de Sistemas

Aula 8 – Acesso a banco de dados

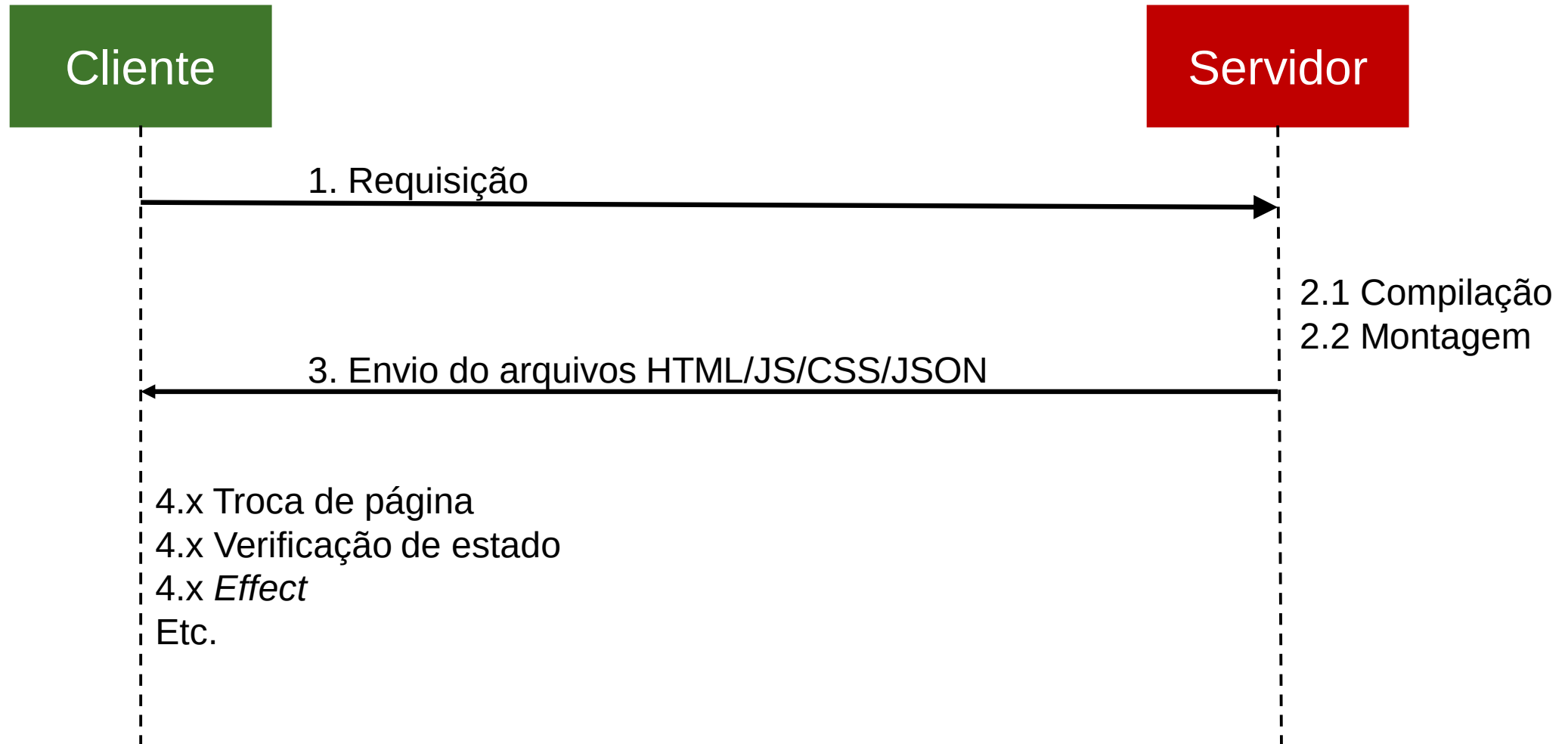
Introdução

Como visto anteriormente, o React passou por uma mudança no objetivo e como ele trabalha a partir da versão 18.

Anteriormente, a ideia era que o React fosse apenas uma biblioteca para desenhar interfaces, independente do *backend* utilizado no *site*.

O código é compilado e empacotado no servidor, sendo seu código JS/HTML/CSS/JSON enviado ao cliente, que faz o restante do processamento no navegador.

Introdução



Introdução

A solução anterior funciona, mas possui duas desvantagens:

- Tamanho do arquivo enviado para o cliente pode ser demasiado grande;
- Segurança.

Introdução

Como todo o código é enviado para ser processado no cliente, o tamanho do pacote enviado pode ser grande para determinadas conexões, fazendo com que o carregamento do *site* seja lento e piorando a experiência do usuário.

Introdução

Em relação à segurança, caso o *site* faça acesso apenas a APIs públicas, sem necessidade de autenticação ou utilização de chaves de segurança, não há problemas, pois estas APIs devem ser acessíveis externamente.

No entanto, caso o *site* faça acesso a APIs privadas, teremos um problema de segurança, pois os dados para acesso restrito à API estarão no código Javascript, enviado ao cliente e, portanto, possível de ser descoberto.

Introdução

Por exemplo, o código a seguir realiza a chamada para uma API, informando a chave de acesso a ela:

```
var args = {  
  method: 'POST',  
  headers: {  
    'Accept': 'application/json',  
    'Content-Type': 'application/json',  
    'x-api-key': 'chave1234'  
  },  
  body: JSON.stringify(data)  
};  
  
fetch('http://enderecodaapi/rota', args).then( /* ... */)
```

Introdução

Como esse código é executado no cliente, e a chave de acesso foi enviada ao cliente, é possível para um invasor obter esta chave e acessar a API, que deveria ter acesso restrito ao *site*.

A solução é fazer a solicitação à API em um componente renderizado no servidor, ao invés de um componente renderizado no cliente.

Componentes no cliente e no servidor

A partir do React 18 (Next 13), é possível definir que um componente vai ser renderizado no servidor, e dessa forma, diminuir o tamanho do arquivo gerado e enviado ao cliente, como era antigamente; Também é possível que esse arquivo seja armazenado no *cache* do servidor, para facilitar a transmissão do mesmo.

Componentes no cliente e no servidor

Por padrão, todo componente criado é renderizado no **servidor**;

Não é obrigatório utilizar renderização no servidor, sendo possível deixar todo o *site* com renderização realizada no cliente.

Componentes no cliente e no servidor

Se for utilizar um dos seguintes itens, obrigatoriamente o componente deve ser renderizado no cliente:

- Interatividade (onClick, por exemplo), estado, efeito, contexto, APIs específicas do navegador (localStorage, por exemplo), *hooks* que dependem de estado, efeitos, etc., e caso queira criar componentes no formato de classe.

Componentes no cliente e no servidor

Caso queira diminuir o tamanho do arquivo gerado, bem como manter informações sensíveis no código, isto deve ser feito em um componente renderizado no servidor.

Acesso a banco de dados

O acesso a banco de dados também deve ser realizado de forma cuidadosa, pois expor os dados do banco de dados para o código executado no cliente não é ideal.

Dessa forma, podemos executar as funções de acesso ao banco de dados em funções que executam no servidor, e evitar que dados sigilosos de acesso ao banco de dados fiquem expostos para o navegador do cliente.

Acesso a banco de dados

Vamos utilizar o Prisma (<https://www.prisma.io>) como uma biblioteca ORM para acesso a dados.

Como banco de dados, vamos utilizar o PostgreSQL em uma instância criada no Docker.

O Prisma funciona com diversos bancos de dados, como SQL Server, MySQL, Postgree, SQLite, etc.

Exemplo

Crie um novo site usando o *framework* Next.js:

1. Baixe a última versão LTS do Node:
<https://nodejs.org/en>
2. Atualize o npm: `npm i -g npm@latest`
3. Crie um novo projeto:
`npx create-next-app@latest`
4. Deixe apenas a primeira linha do arquivo `app/globals.css`

Exemplo

5. Crie a pasta app/livros;
6. Dentro de “app/page.js”, crie um *link* para a rota livros:

```
import Link from "next/link";

export default function Home() {
  return(
    <>
      <Link href="/livros">Livros</Link>
    </>
  )
}
```


Exemplo

7. Crie o arquivo “app/livros/page.js”, com o seguinte conteúdo:

```
'use client';

import { useState } from "react";

export default function Livros() {

  const [busy, setBusy] = useState(false);

  const handleAtualizar = async () => {
    setBusy(true);
    setBusy(false);
  }

  return (
    <div>
      <p>Livros disponíveis</p>
      <button className="bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700 active:bg-blue-800 disabled:bg-blue-200" disabled={busy}
onClick={handleAtualizar}>
        {busy ? " ..... " : "Atualizar"}
      </button>
    </div>
  )
}
```

Acesso a banco de dados

Inicie o Docker, e execute o seguinte comando para iniciar um container do PostgreSQL:

```
docker run --name postgres -e POSTGRES_PASSWORD=senhadobanco -p 5432:5432 -v postgres:/var/lib/postgresql/data -d postgres:17
```

Acesso a banco de dados

Para instalar o prisma, execute os seguintes comando:

```
npm install prisma @types/pg --save-dev
```

```
npm install @prisma/client @prisma/adapter-pg pg dotenv
```

Digite o seguinte comando para criar os arquivos necessários para utilização do prisma:

```
npx prisma init --datasource-provider postgresql --output  
../generated/prisma
```

Acesso a banco de dados

Será criado um arquivo `.env`, com as configurações para acesso ao banco de dados. Este arquivo será lido apenas no servidor.

Será criado um arquivo `prisma.config.ts`, com configurações de acesso do Prisma.

Também será criada uma pasta “prisma”, com o arquivo `schema.prisma`, onde ficará a nossa modelagem de dados.

Configuração do banco de dados

No arquivo .env criado, altere a variável DATABASE_URL para o valor a seguir:

```
DATABASE_URL =  
"postgresql://postgres:senhadobanco@localhost:5432/mydb?schema=public"
```

Atualize o nome do container (postgres), da senha (senhadobanco) e da base de dados (mydb) se necessário.

Modelando os dados

Inicialmente, vamos guardar alguns dados simples de cada livro, como:

- Identificador (inteiro – chave primária e auto incrementável);
- Título (*string* de, no máximo, 200 caracteres);
- Ano de publicação.

Toda modelagem a ser realizada deve ser feita no arquivo `schema.prisma`.

Modelando os dados

Adicione o seguinte conteúdo em schema.prisma:

```
model Livro {  
  id Int @id @default (autoincrement())  
  titulo String  
  anoPublicacao Int  
}
```

O prisma deverá fazer o mapeamento desta *model* para uma tabela na base de dados.

Modelando os dados

Para conectar na base de dados e gerar a alteração desejada, execute o seguinte comando:

```
npx prisma db push
```

Para gerar os arquivos necessários para utilização, execute o seguinte comando:

```
npx prisma generate
```


Acessando os dados

Na raiz do projeto, crie uma pasta chamada **lib**, e dentro dela, um arquivo chamado **prisma.js** com o seguinte conteúdo:

```
import "dotenv/config";
import { PrismaPg } from "@prisma/adapter-pg";
import { PrismaClient } from "../generated/prisma/client";

const connectionString = `${process.env.DATABASE_URL}`;

const adapter = new PrismaPg({ connectionString });
const prisma = new PrismaClient({ adapter });

export { prisma };
```

Acessando os dados

Em “app/livros/page.js”, adicione o seguinte comando para buscar todos os livros na tabela Livro. **Este código não irá funcionar.**

```
import { prisma } from "../../lib/prisma";
```

```
const handleAtualizar = async () => {  
  setBusy(true);  
  const livros =  
prisma.livro.findMany();  
  console.log(livros);  
  setBusy(false);  
}
```

Acessando os dados

Para exibir mensagens ao usuário, utilizaremos a biblioteca Sweet Alert 2.

Faça sua instalação:

```
npm install sweetalert2
```

Faça a sua importação em livros/page.js, e utilize a função **fire** para exibir uma mensagem ao usuário.

Acessando os dados

```
import Swal from 'sweetalert2';

const handleAtualizar = async () => {
  setBusy(true);

  try {
    const livros = prisma.livro.findMany();
    console.log(livros);
  }
  catch {
    Swal.fire({
      text: "Erro ao buscar os dados",
      icon: 'error',
      timer: 3000,
      toast: true,
      position: "top-right",
      showConfirmButton: false
    })
  }
  setBusy(false);
}
```

Este código ainda não funciona!

Acessando os dados

O erro anterior acontece pois devemos utilizar o prisma em um componente no servidor, e não no cliente.

Para isso, dentro da pasta “app/livros”, crie um arquivo chamado actions.js. Ele será responsável pelas funções de acesso ao banco de dados.

Como este arquivo deve ser renderizado no cliente, o iniciaremos com a frase “use server”.

Acessando os dados

Crie uma função chamada Listar, que retornará todos os livros cadastrados na tabela Livro.

```
'use server'
```

```
import { prisma } from "../../lib/prisma";
```

```
export async function Listar() {  
  const livros = await prisma.livro.findMany();  
  return livros;  
}
```

Acessando os dados

Retornando ao arquivo “app/livros/page.js”, atualize a função `handleAtualizar` para fazer uma chamada à função `Listar` criada anteriormente.

Essa função executa no servidor, e nosso cliente aguarda sua execução para poder tomar ações no navegador.

```

'use client';

import { useState } from "react";
import Swal from 'sweetalert2';
import { Listar } from "../actions";

export default function Livros() {

  const [busy, setBusy] = useState(false);

  const handleAtualizar = async () => {
    setBusy(true);

    try {
      const livros = await Listar();
      console.log(livros);
    }
    catch {
      Swal.fire({
        text: "Erro ao buscar os dados",
        icon: 'error',
        timer: 3000,
        toast: true,
        position: "top-right",
        showConfirmButton: false
      })
    }
    setBusy(false);
  }

  return (
    <div>
      <p>Livros disponíveis</p>
      <button className="bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700 active:bg-blue-800 disabled:bg-blue-200"
disabled={busy} onClick={handleAtualizar}>
        {busy ? " ..... " : "Atualizar"}
      </button>
    </div>
  )
}

```


Acessando os dados

Crie um estado **dados** para armazenar os dados vindos da base de dados (inicie com um vetor vazio).

Crie uma tabela no retorno do componente, e o preencha com os dados vindos da base de dados.

```
'use client';

import { useState } from "react";
import Swal from 'sweetalert2';
import { Listar } from "../actions";

export default function Livros() {

  const [busy, setBusy] = useState(false);
  const [dados, setDados] = useState([]);

  const handleAtualizar = async () => {
    setBusy(true);

    try {
      const livros = await Listar();
      setDados(livros);
    }
    catch {
      Swal.fire({
        text: "Erro ao buscar os dados",
        icon: 'error',
        timer: 3000,
        toast: true,
        position: "top-right",
        showConfirmButton: false
      })
    }
    setBusy(false);
  }

  return (
    <div>
      <p>Livros disponíveis</p>
      <button className="bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700 active:bg-blue-800 disabled:bg-blue-200" disabled={busy} onClick={handleAtualizar}>
        {busy ? " ..... " : "Atualizar"}
      </button>
      <table className="w-full">
        <thead>
          <tr>
            <td>Título</td>
            <td>Ano de Publicação</td>
          </tr>
        </thead>
        <tbody>
          {
            dados.map((p) => {
              return (
                <tr key={p.id}>
                  <td>{p.titulo}</td>
                  <td>{p.anoPublicacao}</td>
                </tr>
              )
            })
          }
        </tbody>
      </table>
    </div>
  )
}
```

Listagem na primeira exibição da página

Da forma como está, a página carrega os dados apenas quando o botão atualizar é clicado.

Para carregar também ao iniciar a página:

- Crie um estado **atualizar** para que, sempre que seu valor for **true**, os dados sejam buscados da base de dados;
- Inicie seu estado como **true** para buscar os dados na exibição da página;
- Altere o evento de clique do botão atualizar para alterar esse estado para **true**;
- Crie um efeito que chama a função de atualizar os dados, e altere o estado **atualizar** para **false** após a sua execução.

```
'use client';

import { useEffect, useState } from "react";
import Swal from 'sweetalert2';
import { Listar } from "../actions";

export default function Livros() {

  const [busy, setBusy] = useState(false);
  const [dados, setDados] = useState([]);
  const [atualizar, setAtualizar] = useState(true);

  const handleAtualizar = async () => {
    setBusy(true);

    try {
      const livros = await Listar();
      setDados(livros);
    }
    catch {
      Swal.fire({
        text: "Erro ao buscar os dados",
        icon: 'error',
        timer: 3000,
        toast: true,
        position: "top-right",
        showConfirmButton: false
      })
    }

    setAtualizar(false);
    setBusy(false);
  }

  useEffect(() => {
    if (atualizar)
      handleAtualizar();
  }, [atualizar]);

  return (
    <div>
      <p>Livros disponíveis</p>
      <button className="bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700 active:bg-blue-800 disabled:bg-blue-200" disabled={busy} onClick={() => { setAtualizar(true) }}>
        {busy ? "....." : "Atualizar"}
      </button>
      <table className="w-full">
        <thead>
          <tr>
            <td>Título</td>
            <td>Ano de Publicação</td>
          </tr>
        </thead>
        <tbody>
          {
            dados.map((p) => {
              return (
                <tr key={p.id}>
                  <td>{p.titulo}</td>
                  <td>{p.anoPublicacao}</td>
                </tr>
              )
            })
          }
        </tbody>
      </table>
    </div>
  )
}
```

Inserção de livro

Para cadastrar um novo livro, criaremos um novo componente em `app/livros/novo/page.js`.

Este componente será montado ao clicar em um botão na tela de pesquisa de livros, que deve ter o botão inserido como um Link:

```
<Link className="ml-4 bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700 active:bg-blue-800 disabled:bg-blue-200" href={"/livros/novo"}>Novo</Link>
```

Não esqueça sua importação:

```
import Link from "next/link";
```

```
'use client';

import { useEffect, useState } from "react";
import Swal from 'sweetalert2';
import { Listar } from "../actions";
import Link from "next/link";

export default function Livros() {

  const [busy, setBusy] = useState(false);
  const [dados, setDados] = useState([]);
  const [atualizar, setAtualizar] = useState(true);

  const handleAtualizar = async () => {
    setBusy(true);

    try {
      const livros = await Listar();
      setDados(livros);
    } catch {
      Swal.fire({
        text: "Erro ao buscar os dados",
        icon: 'error',
        timer: 3000,
        toast: true,
        position: "top-right",
        showConfirmButton: false
      })
    }

    setAtualizar(false);
    setBusy(false);
  }

  useEffect(() => {
    if (atualizar)
      handleAtualizar();
  }, [atualizar]);

  return (
    <div>

      <p>Livros disponíveis</p>
      <button className="bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700 active:bg-blue-800 disabled:bg-blue-200" disabled={busy} onClick={() => { setAtualizar(true) }}>
        {busy ? " ....." : "Atualizar"}
      </button>
      <Link className="ml-2 bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700 active:bg-blue-800 disabled:bg-blue-200" href={"/livros/novo"}>Novo</Link>

      <table className="w-full">
        <thead>
          <tr>
            <td>Título</td>
            <td>Ano de Publicação</td>
          </tr>
        </thead>
        <tbody>
          {
            dados.map((p) => {
              return (
                <tr key={p.id}>
                  <td>{p.titulo}</td>
                  <td>{p.anoPublicacao}</td>
                </tr>
              )
            })
          }
        </tbody>
      </table>

    </div>
  )
}
```

Inserção de livro

Instale o React Hook Forms:

```
npm i react-hook-form @hookform/resolvers yup
```

Crie em /livros um arquivo chamado **schema.js**, com o conteúdo a ser vinculado ao formulário de cadastro de livro.

Inserção de livro

```
import * as yup from "yup"

export const LivroSchema = yup.object({
  titulo: yup.string()
    .min(2, 'O título deve possuir, no mínimo, 2 caracteres')
    .max(100, 'O título deve possuir, no máximo, 100 caracteres')
    .required('O título é obrigatório'),
  anoPublicacao: yup.number()
    .integer()
    .min(1500, 'O valor mínimo de ano de publicação é 1500')
    .max(2026, 'O valor máximo de ano de publicação é 1500')
    .required('O ano de publicação é obrigatório')
    .TypeError('O ano de publicação é obrigatório'),
}).required();
```


Inserção de livro

Em `app/livros/actions.js`, crie a função para salvar os dados informados. Faça a validação para garantir que os dados estão conforme o esperado.

```
async function validar(livro) {

  livro.titulo = livro.titulo?.trim();

  if (!livro.titulo || livro.titulo.length < 2 || livro.titulo.length > 100)
    throw new Error('Título inválido');

  if (!livro.anoPublicacao || livro.anoPublicacao < 1500 || livro.anoPublicacao > 2026)
    throw new Error('Ano de publicação inválido');
}

export async function salvar(livro) {

  try {
    await validar(livro);
  }
  catch (e) {
    return { sucesso: false, mensagem: e };
  }

  let resultado = null;
  try {
    resultado = await prisma.livro.create({
      data: livro
    });
  }
  catch {
    return { sucesso: false, mensagem: 'Erro na inserção' };
  }

  return { sucesso: true, dados: resultado, mensagem: 'Inserção realizada com sucesso' };
}
```

Inserção de livro

Na página de criação de livro, crie um formulário para controlar os componentes do livro (título e ano de publicação), bem como um botão para fazer a submissão do formulário.

```
'use client'

import { useState } from "react"
import { useForm } from "react-hook-form"
import { yupResolver } from "@hookform/resolvers/yup"
import { LivroSchema } from "../schema";
import { Salvar } from "../actions";
import Swal from 'sweetalert2';

export default function Novo() {

  const [busy, setBusy] = useState(false);

  const { register, handleSubmit, formState: { errors } } = useForm({
    defaultValues: {
      titulo: '',
      anoPublicacao: '',
    },
    resolver: yupResolver(LivroSchema),
  });

  const onSubmit = async (data) => {
    setBusy(true);
    const resultado = await Salvar(data);
    setBusy(false);

    if (resultado.sucesso) {
      if (resultado.mensagem) {
        Swal.fire({
          text: resultado.mensagem,
          icon: 'success',
          timer: 3000,
          toast: true,
          position: "top-right",
          showConfirmButton: false
        })
      }
    }
    else {
      let mensagem = "Erro ao buscar os dados";
      if (resultado.mensagem)
        mensagem = resultado.mensagem;

      Swal.fire({
        text: mensagem,
        icon: 'error',
        timer: 3000,
        toast: true,
        position: "top-right",
        showConfirmButton: false
      })
    }
  }

  return (
    <div className="mt-2 ml-2">
      <form className=" flex flex-row gap-8" onSubmit={handleSubmit(onSubmit)}>
        <label>
          Titulo
          <input type="text" className="bg-neutral-50 border mx-2 px-2 py-2" {...register("titulo")} />
          <p className="text-sm text-red-600">{errors?.titulo?.message}</p>
        </label>
        <label>
          Ano de publicação
          <input type="number" className="bg-neutral-50 border ml-2 px-2 py-2" {...register("anoPublicacao")} />
          <p className="text-sm text-red-600">{errors?.anoPublicacao?.message}</p>
        </label>
        <button type="submit" className="bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700 active:bg-blue-800 disabled:bg-blue-200 ml-2" disabled={busy}>
          {busy ? " ..... " : "Salvar"}
        </button>
      </form>
    </div>
  )
}
```

Inserção de livro

Atualize a página de inserção de livro para fazer a chamada para a função anterior. Em caso de sucesso, usaremos o *hook* **useRouter** para redirecionar o usuário para a tela anterior, passando um parâmetro que informa que uma mensagem de sucesso deve ser exibida.

```
use client'

import { useState } from "react"
import { useForm } from "react-hook-form"
import { yupResolver } from "@hookform/resolvers/yup"
import { LivroSchema } from "../schema";
import { Salvar } from "../actions";
import Swal from 'sweetalert2';
import { useRouter } from "next/navigation";

export default function Novo() {

  const [busy, setBusy] = useState(false);
  const router = useRouter();

  const { register, handleSubmit, formState: { errors } } = useForm({
    defaultValues: {
      titulo: '',
      anoPublicacao: '',
    },
    resolver: yupResolver(LivroSchema),
  });

  const onSubmit = async (data) => {
    setBusy(true);
    const resultado = await Salvar(data);
    setBusy(false);

    if (resultado.sucesso) {
      if (resultado.mensagem) {
        Swal.fire({
          text: resultado.mensagem,
          icon: 'success',
          timer: 3000,
          toast: true,
          position: "top-right",
          showConfirmButton: false
        })
      }
      router.push("/livros");
    }
    else {
      let mensagem = "Erro ao buscar os dados";
      if (resultado.mensagem)
        mensagem = resultado.mensagem;

      Swal.fire({
        text: mensagem,
        icon: 'error',
        timer: 3000,
        toast: true,
        position: "top-right",
        showConfirmButton: false
      })
    }
  }

  return (
    <div className="mt-2 ml-2">
      <Form className=" flex flex-row gap-8" onSubmit={handleSubmit(onSubmit)}>
        <label>
          Titulo
          <input type="text" className="bg-neutral-50 border mx-2 px-2 py-2" {...register("titulo")} />
          <p className="text-sm text-red-600">{errors?.titulo?.message}</p>
        </label>
        <label>
          Ano de publicação
          <input type="number" className="bg-neutral-50 border ml-2 px-2 py-2" {...register("anoPublicacao")} />
          <p className="text-sm text-red-600">{errors?.anoPublicacao?.message}</p>
        </label>
        <button type="submit" className="bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700 active:bg-blue-800 disabled:bg-blue-200 ml-2" disabled={busy}>
          {busy ? " ..... " : "Salvar"}
        </button>
      </form>
    </div>
  )
}
```

Edição de livro

Para informar que um livro deve ser editado, exibiremos um *link* para a sua edição ao lado dos seus dados, e quando o usuário clicar em um destes *links*, ele será redirecionado para a página correta, de acordo com o id do livro selecionado.

```
'use client';

import { use, useEffect, useState } from "react";
import Swal from 'sweetalert2';
import { Listar } from "../actions";
import Link from "next/link";

export default function Livros() {

  const [busy, setBusy] = useState(false);
  const [dados, setDados] = useState([]);
  const [atualizar, setAtualizar] = useState(true);

  const handleAtualizar = async () => {
    setBusy(true);

    try {
      const livros = await Listar();
      setDados(livros);
    }
    catch {
      Swal.fire({
        text: "Erro ao buscar os dados",
        icon: 'error',
        timer: 3000,
        toast: true,
        position: "top-right",
        showConfirmButton: false
      })
    }

    setAtualizar(false);
    setBusy(false);
  }

  useEffect(() => {
    if (atualizar)
      handleAtualizar();
  }, [atualizar]);

  return (
    <div>

      <p>Livros disponíveis</p>
      <button className="bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700 active:bg-blue-800 disabled:bg-blue-200" disabled={busy} onClick={() => { setAtualizar(true) }}>
        {busy ? " ....." : "Atualizar"}
      </button>
      <Link className="ml-2 bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700 active:bg-blue-800 disabled:bg-blue-200" href={"/livros/novo"}>Novo</Link>

      <table className="w-full">
        <thead>
          <tr>
            <td>Título</td>
            <td>Ano de Publicação</td>
          </tr>
        </thead>
        <tbody>
          {
            dados.map((p) => {
              return (
                <tr key={p.id}>
                  <td>{p.titulo}</td>
                  <td>{p.anoPublicacao}</td>
                  <td><Link href={` /livros/editar/${p.id}`} className="text-blue-600 pl-2">Editar</Link></td>
                </tr>
              )
            })
          }
        </tbody>
      </table>

    </div>
  )
}
```


Edição de livro

Em app/livros/action.js, crie a função para obter os dados de um livro a partir de seu id:

```
export async function obter(id) {  
  const livro = prisma.livro.findUnique({  
    where: {  
      id: id  
    }  
  });  
  
  return livro;  
}
```

Edição de livro

Em `app/livros/action.js`, crie também a função para atualizar os dados de um livro recebido.

```
export async function Atualizar(livro) {  
  try {  
    await validar(livro);  
  }  
  catch (e) {  
    return { sucesso: false, mensagem: e };  
  }  
  
  let resultado = null;  
  try {  
    resultado = await prisma.livro.update({  
      where: {  
        id: livro.id  
      },  
      data: { ...livro, id: undefined }  
    });  
  }  
  catch {  
    return { sucesso: false, mensagem: 'Erro na atualização' };  
  }  
  
  return { sucesso: true, dados: resultado, mensagem: 'Atualização realizada com sucesso' };  
}
```

Edição de livro

Crie a pasta `app/livros/editar/[id]`, e o arquivo `page.js` dentro dela.

Esta página, ao ser aberta, deve verificar o id informado e carregar os dados do livro obtido através da base de dados.

Também deve possuir um botão salvar que, quando clicado, atualiza os dados deste livro e redireciona o usuário à tela de listagem de livros.

```
'use client'
import { use, useEffect, useState } from "react"
import { useForm } from "react-hook-form"
import { yupResolver } from "@hookform/resolvers/yup"
import { Livroschema } from "../schema";
import { Atualizar, Obter } from "../actions";
import Swal from 'sweetalert2';
import { useRouter } from "next/navigation";

export default function Novo({ params }) {

  const [busy, setBusy] = useState(false);
  const router = useRouter();
  const { id } = use(params);

  const { register, handleSubmit, reset, formState: { errors } } = useForm({
    defaultValues: {
      titulo: '',
      anoPublicacao: '',
    },
    resolver: yupResolver(Livroschema),
  });

  const onSubmit = async (data) => {
    setBusy(true);
    data.id = parseInt(id);
    const resultado = await Atualizar(data);
    setBusy(false);

    if (resultado.sucesso) {
      if (resultado.mensagem) {
        Swal.fire({
          text: resultado.mensagem,
          icon: 'success',
          timer: 3000,
          toast: true,
          position: "top-right",
          showConfirmButton: false
        })
      }
      router.push("/livros");
    }
    else {
      let mensagem = "Erro ao buscar os dados";
      if (resultado.mensagem)
        mensagem = resultado.mensagem;

      Swal.fire({
        text: mensagem,
        icon: 'error',
        timer: 3000,
        toast: true,
        position: "top-right",
        showConfirmButton: false
      })
    }
  }

  const handleCarregar = async () => {
    let atual = null;
    try {
      setBusy(true);
      const idInt = parseInt(id);
      atual = await Obter(idInt);
    }
    catch {
      setBusy(false);
      router.replace("/livros");
    }
    setBusy(false);
    if (atual) {
      reset({
        titulo: atual.titulo,
        anoPublicacao: atual.anoPublicacao
      })
    }
    else
      router.replace("/livros");
  }

  useEffect(() => {
    if (id) {
      handleCarregar();
    }
  }, [id]);

  return (
    <div className="mt-2 ml-2">
      <form className=" flex flex-row gap-8" onSubmit={handleSubmit(onSubmit)}>
        <label>
          Titulo
          <input type="text" className="bg-neutral-50 border mx-2 px-2 py-2" {...register("titulo")} />
          <p className="text-sm text-red-600">{errors?.titulo?.message}</p>
        </label>
        <label>
          Ano de publicação
          <input type="number" className="bg-neutral-50 border ml-2 px-2 py-2" {...register("anoPublicacao")} />
          <p className="text-sm text-red-600">{errors?.anoPublicacao?.message}</p>
        </label>
        <button type="submit" className="bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700 active:bg-blue-800 disabled:bg-blue-200 ml-2" disabled={busy}>
          {busy ? " ..... " : "salvar"}
        </button>
      </form>
    </div>
  )
}
```

Remoção de livro

Em app/livros/actions.js, crie a função para remover o livro com o id informado:

```
export async function Remover(id) {  
  let resultado = null;  
  
  try {  
    resultado = await prisma.livro.delete({  
      where: {  
        id: id  
      }  
    });  
  } catch { }  
  
  if (resultado)  
    return { sucesso: true };  
  else  
    return { sucesso: false };  
}
```

Remoção de livro

Em `app/livros/page.js`, adicione a coluna para remover o livro informado chamando a função criada anteriormente. Em caso de sucesso, atualizar a lista e exibir uma mensagem de sucesso para o usuário. Em caso de erro, exibir uma mensagem de erro e não atualizar a lista.

```
use client';

import { use, useEffect, useState } from "react";
import Swal from 'sweetalert2';
import { Listar, Remover } from "../actions";
import Link from "next/link";

export default function Livros() {

  const [busy, setBusy] = useState(false);
  const [dados, setDados] = useState([]);
  const [atualizar, setAtualizar] = useState(true);

  const handleAtualizar = async () => {
    setBusy(true);

    try {
      const livros = await Listar();
      setDados(livros);
    } catch {
      Swal.fire({
        text: "Erro ao buscar os dados",
        icon: 'error',
        timer: 3000,
        toast: true,
        position: "top-right",
        showConfirmButton: false
      })
    }

    setAtualizar(false);
    setBusy(false);
  }

  const handlerRemover = async (id) => {
    const resposta = confirm('Deseja realmente remover este livro?');
    if (resposta) {
      setBusy(true);
      const resultado = await Remover(id);
      if (resultado.sucesso) {
        setAtualizar(true);
        Swal.fire({
          text: "Livro removido com sucesso",
          icon: 'success',
          timer: 3000,
          toast: true,
          position: "top-right",
          showConfirmButton: false
        })
      }
    } else {
      Swal.fire({
        text: "Erro ao remover o livro informado",
        icon: 'error',
        timer: 3000,
        toast: true,
        position: "top-right",
        showConfirmButton: false
      })
    }
    setBusy(false);
  }

  useEffect(() => {
    if (atualizar)
      handleAtualizar();
  }, [atualizar]);

  return (
    <div>

      <p>Livros disponíveis</p>
      <button className="bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700 active:bg-blue-800 disabled:bg-blue-200" disabled={busy} onClick={() => { setAtualizar(true) }}>
        {busy ? "....." : "Atualizar"}
      </button>
      <Link className="ml-2 bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700 active:bg-blue-800 disabled:bg-blue-200" href={"/livros/novo"}>Novo</Link>

      <table className="w-full">
        <thead>
          <tr>
            <td>Título</td>
            <td>Ano de Publicação</td>
          </tr>
        </thead>
        <tbody>
          {
            dados.map((p) => {
              return (
                <tr key={p.id}>
                  <td>{p.titulo}</td>
                  <td>{p.anoPublicacao}</td>
                  <td><Link href={"/livros/editar/${p.id}"} className="text-blue-600 pl-2">Editar</Link></td>
                  <td><a href="#" className="text-red-600 pl-2" onClick={() => { handlerRemover(p.id) }}>Remover</a></td>
                </tr>
              )
            })
          }
        </tbody>
      </table>

    </div>
  )
}
```